

Séance 1 : Fondations du Real-Time Data Engineering

Données, événements, flux, temporalité et introduction à Apache Kafka

Dr. Khalil Haddaoui

- ➊ Motivation : pourquoi le temps réel ?
- ➋ Données, événements, flux : changement de paradigme
- ➌ Batch processing : définition, intuition, forces et limites
- ➍ Streaming processing : principes et différences profondes
- ➎ Flux d'événements : structure, désordre, latence
- ➏ Temporalité : Event Time, Ingestion Time, Processing Time
- ➐ Architecture d'un pipeline temps réel moderne
- ➑ Apache Kafka : modèle du log distribué et concepts essentiels
- ➒ Synthèse générale puis TP

Objectifs pédagogiques de la journée

À la fin de cette journée, vous serez capable de :

- expliquer **pourquoi** le temps réel est devenu crucial dans de nombreux domaines ;
- distinguer clairement **batch** et **streaming**, au-delà de l'idée “plus rapide” ;
- décrire ce qu'est un **événement**, un **flux** et la notion de **latence** ;
- comprendre la différence entre **Event Time**, **Ingestion Time** et **Processing Time** ;
- dessiner l'architecture logique d'un **pipeline temps réel** moderne ;
- présenter les **concepts essentiels de Kafka** : topics, partitions, offsets, consumer groups.

Fil conducteur

On part du **besoin métier** (“pourquoi?”), puis on construit progressivement les **outils techniques** (“comment?”).

Un monde devenu “temps réel”

Les systèmes modernes produisent des événements en continu :

- **Finance** : transactions carte bancaire, ordres de bourse, flux HFT.
- **E-commerce** : clics, vues produit, abandons de panier.
- **IoT industriel** : température, vibrations, pression, consommation.
- **Réseaux sociaux** : posts, likes, commentaires, partages.
- **Cloud / DevOps** : logs applicatifs, métriques CPU / RAM / latence.

Message fondamental

Le Real-Time Data Engineering permet d'analyser et d'agir **pendant que les événements se produisent**, et non seulement après coup.

Exemples concrets de besoins temps réel

- **Détection de fraude :**

- identifier une transaction suspecte *avant* la validation,
- bloquer la carte immédiatement si nécessaire.

- **Recommandation en ligne :**

- proposer un produit pertinent pendant la navigation,
- s'adapter en fonction des clics de l'utilisateur.

- **Monitoring d'infrastructure :**

- détecter un pic CPU ou une latence anormale,
- déclencher une alerte ou un scaling automatique.

- **Logistique / transport :**

- suivre les véhicules en direct,
- recalculer les itinéraires en fonction du trafic.

Idée clé

Quand la **valeur de la donnée diminue très vite avec le temps**, le traitement batch n'est plus suffisant.

Analogie : contrôle régulier vs surveillance vitale

Batch = Médecin de contrôle

- Bilan annuel, analyses faites après la prise de sang.
- Le médecin observe des résultats **déjà figés**.
- On prend des décisions **avec du recul**, pas dans l'urgence.

Streaming = Urgences / Réanimation

- Monitoring continu : ECG, tension, saturation en oxygène.
- Le médecin agit en fonction de ce qui se passe **maintenant**.
- Les décisions sont **immédiates**, parfois à la milliseconde près.

Parallèle

Le streaming correspond aux situations nécessitant **surveillance permanente et réaction immédiate**.

Question de réflexion

- Pour chacun des cas suivants, quelle latence est acceptable ?
 - surveillance d'un **pacemaker** ;
 - mise à jour des **classements** sur une plateforme d'e-learning ;
 - calcul du **résultat annuel** d'une entreprise ;
 - génération d'un **rapport réglementaire** trimestriel.

Discussion

L'objectif n'est pas que tout soit temps réel, mais d'identifier **où** le temps réel apporte un **gain décisif**.

Données classiques vs données événementielles

Données classiques (tables, fichiers) :

- organisation en lignes / colonnes,
- ensemble **fini** : on sait quand la collecte est terminée,
- exemples : fichiers CSV, tables SQL, export d'un CRM.

Données événementielles :

- chaque enregistrement = un **événement ponctuel dans le temps**,
- le flux d'événements peut être **théoriquement infini**,
- exemples : clic, transaction, mesure capteur, log système.

Changement de paradigme

On ne pense plus seulement en “tables statiques” mais en **flux qui s'écoulent** dans le temps.

Événement : définition intuitive

Un **événement** est une petite structure de données qui décrit :

- **quoi** s'est passé (type d'événement),
- **qui** est concerné (utilisateur, client, capteur),
- **quand** cela s'est produit (timestamp),
- **détails** associés (montant, URL, coordonnées, etc.).

Exemples d'événements

- `page_view(user=42, url="/home", time=10:01:23)`
- `transaction(card=XXXX, amount=120.50, time=10:01:25)`
- `sensor(id=7, temp=86.2, time=10:01:30)`

Exemple : navigation d'un utilisateur

Séquence typique d'un parcours e-commerce :

- `page_view("/home")`
- `search("ordinateur")`
- `page_view("/product/42")`
- `click("add_to_cart", product=42)`
- `checkout(order_id=1234, amount=1299.99)`

Vision “flux”

Ces événements forment un **flux chronologique** pour un utilisateur, et un **flux global** quand on agrège tous les utilisateurs.

Mini-exercice (papier)

Question : Vous observez un log d'événements pour un site :

- `search("ordinateur")`
- `page_view("/home")`
- `checkout(order_id=1234)`

- 1 Proposez un **ordonnancement plausible** en temps réel (Event Time).
- 2 Quelles **informations métier** peut-on déduire de cette séquence ?
- 3 Que perdez-vous si vous mélangez ces événements dans une **table classique** sans ordre ni timestamp ?

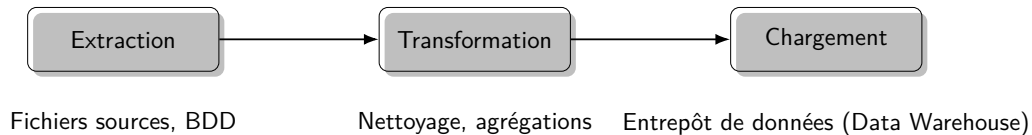
Définition

Le **batch processing** consiste à traiter un ensemble **fini** de données en exécutant un calcul global (ou une série de calculs) sur ce jeu de données, en général selon une planification (une fois par jour, par heure, etc.).

Exemples

- Rapport hebdomadaire de ventes par magasin.
- Calcul du P&L mensuel d'un portefeuille.
- Entraînement d'un modèle de Machine Learning sur les données de l'année passée.

Pipeline batch classique (ETL)



Avantages majeurs

- On dispose de **toutes** les données avant de lancer le calcul.
- Possibilité de :
 - trier, filtrer, nettoyer,
 - relire les données plusieurs fois,
 - utiliser des algorithmes complexes.
- Très adapté à :
 - l'analyse historique,
 - les rapports réglementaires,
 - l'entraînement de modèles sur de grands volumes.

Idée

Le batch = **prendre du recul** sur un historique figé.

Limites principales

- **Latence élevée :**
 - si le batch tourne une fois par jour, la donnée peut avoir 24h de retard,
 - impossible de réagir à une fraude en quelques secondes.
- **Pas de visibilité continue :**
 - on voit “ce qui s’est passé”, pas “ce qui se passe maintenant”.
- **Modèle de données fini :**
 - le traitement suppose un ensemble de données clos,
 - peu adapté à des flux qui ne s’arrêtent jamais (capteurs, logs).

Conséquence

Pour certains cas d’usage (fraude, recommandations en ligne, monitoring), le batch est **structurellement insuffisant**.

Pause conceptuelle : batch

- Dans votre contexte (finance / data science / IT), citez :
 - un exemple où le **batch est suffisant** voire idéal ;
 - un exemple où le batch est clairement **trop lent**.
- Quels sont les **risques métier** liés à une latence trop grande ?

Transition

Nous allons maintenant voir pourquoi le **streaming** n'est pas juste "faire la même chose, mais plus vite".

Streaming processing : définition

Définition

Le **stream processing** consiste à traiter un **flux potentiellement infini** d'événements au fur et à mesure de leur arrivée, avec des contraintes de **latence faible** et la nécessité de maintenir un **état interne** (state) pour agréger l'information.

Différence clé

Le streaming n'est pas un batch "plus rapide" : c'est un **modèle de calcul fondamentalement différent**.

Batch vs Streaming : comparaison structurée

Aspect	Batch	Streaming
Données	Ensemble fini	Flux infini
Moment du traitement	Après collecte	Pendant la collecte
Latence	Minutes, heures, jours	Millisecondes, secondes
Mémoire	Historique complet	État résumé (fenêtres, state)
Objectif	Analyses historiques	Réaction temps réel
Gestion du temps	Temps de traitement	Temps d'événement (Event Time)

Exemple : alerte température machine industrielle

- Un capteur envoie une mesure toutes les 2 secondes.
- On surveille un seuil critique : 90°C.
- On veut être alerté :
 - si la température dépasse 90°C,
 - ou si elle augmente trop rapidement (ex : +10°C en 30s).

Batch vs Streaming

- Batch : on voit le problème **après coup** (trop tard).
- Streaming : on intervient **pendant la montée en température**.

Structure générique d'un événement

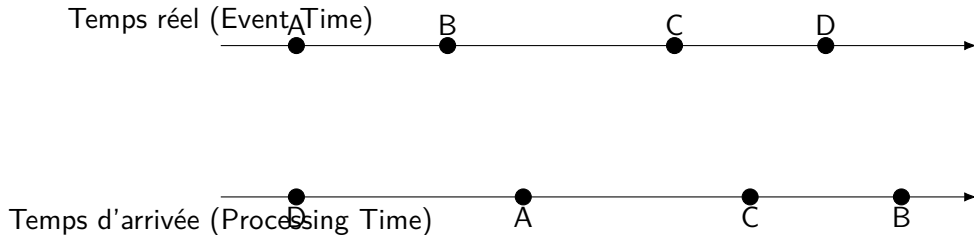
On peut modéliser un événement par une structure clé-valeur :

Champ	Rôle
type	type d'événement (clic, transaction, mesure, log, ...)
timestamp	moment où il s'est produit (Event Time)
key	identifiant logique (user_id, card_id, sensor_id, ...)
value	payload : montant, URL, mesure, etc.

Point crucial

Le **timestamp d'événement** est souvent différent du moment où le système reçoit l'événement.

Désordre naturel des événements



Observation

Les événements A, B, C, D ne sont **pas reçus dans l'ordre** où ils se sont réellement produits.

Pourquoi les événements arrivent-ils en désordre ?

Causes fréquentes :

- **Réseaux mobiles** : latences variables, pertes, reconnections.
- **Buffers** : navigateurs, OS, proxies peuvent retarder certains paquets.
- **Retries automatiques** :
 - une requête échoue, est renvoyée plus tard.
- **Distance géographique** :
 - data centers éloignés, routes réseau différentes.
- **Charge du système** :
 - files d'attente internes, micro-pannes, backpressure.

Conséquence

Toute statistique basée uniquement sur l'ordre d'arrivée **ignore la réalité temporelle du métier.**

Latence : notion essentielle

Définition (intuitive)

La **latence** d'un système temps réel est le délai entre :

- le moment où l'événement se produit,
- le moment où le système réagit (alerte, recommandation, écriture en base).

Exemples :

- Détection de fraude : quelques millisecondes / secondes.
- Monitoring technique : quelques secondes.
- Reporting interne : minutes acceptables.

Compromis

Plus on veut une latence faible, plus :

- l'architecture doit être optimisée,
- on doit accepter des compromis sur la complexité des traitements.

Trois notions de temps à distinguer

① Event Time :

- moment où l'événement s'est réellement produit, du point de vue métier,
- ex : heure exacte du paiement par l'utilisateur.

② Ingestion Time :

- moment où l'événement entre dans la plateforme (Kafka, par ex.),
- dépend du réseau, des buffers, etc.

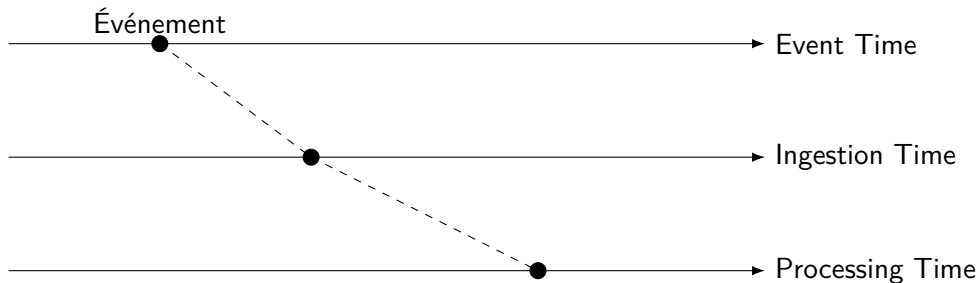
③ Processing Time :

- moment où le job de traitement l'exécute effectivement,
- dépend de la charge, des ressources, de la file interne.

À retenir

Pour les statistiques métier (taux de clics par minute, volumétrie par heure, etc.), c'est l'**Event Time** qui est pertinent.

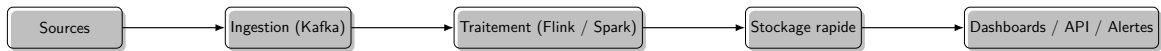
Schéma temporel simplifié



Interprétation

Entre l'événement et sa prise en compte par le système, plusieurs délais techniques s'ajoutent.

Vue d'ensemble d'un pipeline temps réel



Exemples de sources :

- Applications web / mobiles (flux de clics, logs HTTP).
- Systèmes de paiement (transactions).
- Capteurs IoT (mesures physiques).
- Systèmes internes (événements métier, logs).

Rôle de la couche d'ingestion

- absorber de grands volumes d'événements,
- les **bufferiser** et les **rendre disponibles** aux applications consommatrices,
- offrir des garanties de durabilité et d'ordre (partiel).

Rôle de la couche de traitement :

- filtrer les événements inutiles,
- enrichir (join avec d'autres flux ou données de référence),
- agréger (comptages, moyennes, fenêtres temporelles),
- détecter des patterns (fraude, anomalies, comportements).

Exemples d'outils :

- Apache Flink (streaming-first),
- Spark Structured Streaming (micro-batch),
- Kafka Streams (bibliothèque embarquée côté microservice).

Stockages typiques :

- **Redis** : cache rapide pour scores, alertes, compteurs.
- **Cassandra** : historique temps réel clé-valeur.
- **ClickHouse** : requêtes analytiques rapides.
- **Elasticsearch** : recherche plein texte, logs.

Consommateurs :

- dashboards (Grafana, Kibana),
- applications front (API REST / GraphQL),
- systèmes d'alerting (emails, SMS, Slack, etc.).

Pourquoi Kafka est devenu central ?

- Débit très élevé (millions de messages par seconde).
- Stockage persistant des messages sur disque.
- Possibilité de **rejouer** les événements (replay).
- Tolérance aux pannes : réplication, cluster de brokers.
- Partitionnement $\Rightarrow$ scalabilité horizontale.

Idée clé

Kafka n'est pas une simple file de messages : c'est un **log distribué immuable et partitionné**.

Topic "paiements"

Partition 0

Partition 1

Partition 2

Ordre dans Kafka

L'ordre est garanti **à l'intérieur d'une partition**, mais pas entre partitions différentes.

Offsets : positions dans le log

Chaque message dans une partition reçoit un **offset** croissant :

$$\text{offset} = 0, 1, 2, 3, \dots$$

Propriétés importantes :

- lecture séquentielle très efficace,
- reprise exacte après une panne (en mémorisant l'offset),
- possibilité de **relire** l'historique (replay).

Reprise après panne

Si un consumer tombe après avoir traité l'offset 123, il peut redémarrer et continuer à partir de 124.

Rôle :

- envoyer des messages vers un topic,
- choisir la partition (via une clé ou un hash),
- contrôler le niveau de fiabilité de l'écriture (acks).

Paramètre acks :

- acks=0 : le producer n'attend aucune confirmation (rapide mais risqué).
- acks=1 : confirmation du leader de partition.
- acks=all : confirmation du leader + réplicas (plus fiable).

Consumers : lire depuis Kafka

Rôle :

- lire séquentiellement les messages d'une ou plusieurs partitions,
- traiter, transformer, envoyer vers d'autres systèmes.

Gestion de la position (offset) :

- l'offset lu peut être **commité** périodiquement,
- permet de reprendre au bon endroit après un redémarrage.

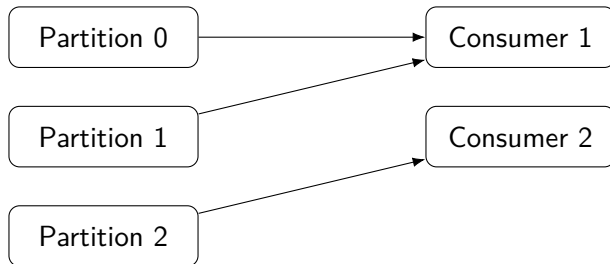
Découplage

Producers et consumers sont découplés dans le temps : un consumer peut arriver plus tard et relire l'historique.

Consumer groups : paralléliser la consommation

Principe :

- un **consumer group** = un ensemble d'instances qui coopèrent pour lire un même topic,
- Kafka garantit qu'une partition donnée est lue par **au plus un consumer** dans un groupe.



Conséquence

On peut augmenter le nombre de consumers pour augmenter la **parallélisation**.

Garanties de livraison (introduction)

Trois niveaux conceptuels :

- **At-most-once** :
 - chaque message est traité au plus une fois,
 - risque de **perte** de messages.
- **At-least-once** :
 - chaque message est traité **au moins une fois**,
 - possible **duplication** (traitement multiple).
- **Exactly-once** :
 - chaque message est traité **exactement une fois**,
 - nécessite une coordination avec le système de traitement (ex : Flink).

Point important

Le traitement de niveau **exactly-once** est complexe à mettre en œuvre, mais essentiel pour certains cas métier (ex : comptabilité).

Nous avons vu :

- les **motivations** du temps réel dans le monde moderne (finance, IoT, web),
- la différence conceptuelle **batch vs streaming** (fini vs infini, historique vs temps réel),
- la nature des **flux d'événements** (structure, désordre, latence),
- la **temporalité** Event / Ingestion / Processing Time,
- l'architecture d'un **pipeline temps réel** moderne,
- les fondations d'**Apache Kafka** :
 - topics, partitions, offsets,
 - producers, consumers, consumer groups,
 - notion de garanties de livraison.

Dans le **Notebook**, vous allez :

- manipuler des flux simulés d'événements (clics, transactions),
- observer **concrètement** :
 - le désordre entre Event Time et Processing Time,
 - l'impact de la latence,
 - des scénarios batch vs streaming.
- construire de petites fonctions Python pour :
 - agréger des événements par fenêtres de temps,
 - comparer des approches naïves vs “correctes” au niveau temporel.

Objectif

Passer de la compréhension **théorique** à la compréhension **opérationnelle**.